

Python Backend Developer + Agentic AI

Complete Roadmap — 5 Years Experience Level

Preparation Guide for Senior Python + AI Engineer Role

8 Phases | 17 Weeks | Interview Ready

8
Phases

250+
Topics

4
Projects

10 Wks
Timeline

Designed for developers with 4+ years of Python backend experience
transitioning into Agentic AI engineering.

Table of Contents

PHASE 1	Python Strong Foundation	Week 1-2
	— 1.1 Python Core — 1.2 Python Advanced — 1.3 Python Tooling [NEW]	
PHASE 2	Backend Core	Week 3-5
	— 2.1 FastAPI — 2.2 Django + DRF — 2.3 Database — 2.4 Async Task Queue	
PHASE 3	System Design + Architecture	Week 6-7
	— 3.1 API Design — 3.2 Microservices — 3.3 Security — 3.4 DevOps — 3.5 gRPC [NEW]	
PHASE 4	AI/ML Fundamentals	Week 8-9
	— 4.1 LLM Basics — 4.2 OpenAI API — 4.3 Anthropic Claude API — 4.4 LiteLLM [NEW] — 4.5 Instructor Library [NEW]	
PHASE 5	Agentic AI Core	Week 10-12
	— 5.1 LangChain — 5.2 LangGraph — 5.3 RAG — 5.4 MCP — 5.5 Agent Patterns — 5.6 CrewAI [NEW] — 5.7 Agent Memory [NEW] — 5.8 Web Search Tools [NEW] — 5.9 DSPy [NEW]	
PHASE 6	Advanced Agentic AI	Week 13-15
	— 6.1 Vector Databases — 6.2 Production AI Systems — 6.3 AI APIs — 6.4 Specialized AI	

- 6.5 LLM Fine-tuning [NEW]
- 6.6 GraphRAG [NEW]
- 6.7 Enterprise AI Platforms [NEW]
- 6.8 AI Testing [NEW]

PHASE 7

Projects — Build & Deploy

Week 16

- Project 1: Personal AI Assistant with MCP
- Project 2: RAG Document Q&A; System
- Project 3: Multi-Agent Code Review System
- Project 4: Production AI API (Mini SaaS)

PHASE 8

Interview Preparation

Week 17

- 8.1 System Design for AI
- 8.2 Coding Patterns
- 8.3 Behavioral Questions
- 8.4 Key Technical Concepts

Also included: [Priority Stack \(What Gets You Hired\)](#) • [Resource Stack](#) • [10-Week Timeline Summary](#)

1.1 Python Core

- Data Types, Variables, Type Hints
- List, Dict, Set, Tuple — comprehensions
- Functions — *args, **kwargs, decorators, closures
- Classes — OOP, inheritance, dunder methods
- Iterators, Generators, Context Managers
- Exception Handling — custom exceptions
- Async/Await — asyncio, event loop
- Threading vs Multiprocessing vs Async

1.2 Python Advanced

- Pydantic v2 — data validation, models
- Dataclasses vs Pydantic
- Abstract Base Classes
- Design Patterns — Singleton, Factory, Repository, Strategy
- Memory management, GIL
- Type annotations — TypeVar, Generic, Protocol

1.3 Python Tooling + NEW

Modern Python project setup — every production team uses these.

- Poetry / uv — modern package management [NEW]
- Virtual environments — venv, pyenv [NEW]
- SQLAlchemy 2.0 — async ORM (FastAPI standard) [NEW]
- Alembic — database migrations (non-Django projects) [NEW]
- pre-commit hooks — code quality automation [NEW]
- Ruff — fast Python linter (black + flake8 replacement) [NEW]
- mypy — static type checking [NEW]

2.1 FastAPI

- Routing, Path/Query params, Request body
- Dependency Injection system
- Middleware — CORS, logging, timing
- Background Tasks
- WebSockets
- File upload/download
- OpenAPI/Swagger auto docs
- Lifespan events (startup/shutdown)
- Testing with pytest + httpx
- SQLAlchemy 2.0 async with FastAPI
- Alembic migrations with FastAPI
- Repository pattern with FastAPI + SQLAlchemy
- Custom exception handlers + error response schema
- API response standardization (BaseResponse pattern)

2.2 Django + DRF (Deep Dive)

- ORM — select_related, prefetch_related, annotate, aggregate
- Custom managers and querysets
- Signals — pre_save, post_save
- Middleware writing
- Management commands
- DRF — ViewSets, Routers, Serializers
- DRF — Permissions, Throttling, Pagination
- DRF — Custom authentication
- Django Channels (WebSocket)

2.3 Database

- PostgreSQL — indexes, EXPLAIN ANALYZE, query optimization
- Transactions — ACID, isolation levels
- Connection pooling — pgBouncer
- PostGIS — geospatial queries
- Redis — caching, pub/sub, sorted sets, streams
- Database migrations strategy
- Read replicas, sharding basics
- SQLAlchemy 2.0 — async sessions, relationship loading

- Database soft deletes pattern
- Optimistic vs Pessimistic locking
- TimescaleDB basics — time-series data (AI logs)

2.4 Async Task Queue

- Celery — tasks, chains, chords, groups
- Celery beat — periodic tasks
- RabbitMQ vs Redis as broker
- Task retry logic, exponential backoff
- Monitoring — Flower dashboard
- AWS SQS integration

3.1 API Design

- REST best practices
- Versioning strategies
- Rate limiting — token bucket, sliding window
- Idempotency keys
- Pagination — cursor vs offset
- Webhook design
- API Gateway patterns

3.2 Microservices

- Service decomposition
- Inter-service communication — REST, gRPC, message queue
- Circuit breaker pattern
- Saga pattern — distributed transactions
- Event-driven architecture
- CQRS basics

3.3 Security

- JWT — access + refresh tokens
- OAuth2 flows
- RBAC — role-based access control
- API key management
- Rate limiting per user/IP
- Input validation, SQL injection prevention
- HTTPS, CORS, CSP headers

3.4 DevOps

- Docker — multi-stage builds, compose
- Nginx — reverse proxy, load balancing
- GitHub Actions — CI/CD pipeline
- AWS — EC2, S3, RDS, SQS/SNS, IAM roles
- Prometheus + Grafana — metrics
- ELK / Loki — logging
- Health checks, readiness probes
- Kubernetes — Deployments, Services, ConfigMaps, Secrets
- Helm charts basics

- Terraform — IaC basics (AWS resources provisioning)
- AWS Bedrock — managed LLM service
- AWS Lambda + API Gateway — serverless AI endpoints
- Horizontal Pod Autoscaling — AI workloads

3.5 gRPC + NEW

Standard for high-performance inter-service communication in microservices.

- Protocol Buffers — .proto file definition [NEW]
- gRPC server/client in Python [NEW]
- Unary vs streaming RPCs [NEW]
- gRPC vs REST vs GraphQL — when to use which [NEW]
- grpcio + grpcio-tools [NEW]
- gRPC with FastAPI (grpc-gateway pattern) [NEW]

4.1 LLM Basics

- How LLMs work — tokens, context window
- Prompt engineering — zero-shot, few-shot, chain-of-thought
- Temperature, top-p, max tokens
- System prompts vs user prompts
- Token counting and cost management
- Streaming responses
- Function calling / Tool use
- Prompt caching — Anthropic + OpenAI (cost reduction)
- Structured outputs — JSON Schema enforcement
- Extended thinking — Claude reasoning models
- Context window management — sliding window, summarization
- Model routing — fast model vs powerful model decision
- Multimodal — text + image + audio combined inputs

4.2 OpenAI API

- Chat Completions API
- Embeddings API
- Vision API (GPT-4V)
- Batch API
- Rate limiting — TPM/RPM management
- Retry logic with exponential backoff
- Cost optimization strategies
- Structured outputs API (not just JSON mode)
- Assistants API vs Chat Completions — when to use
- OpenAI fine-tuning API

4.3 Anthropic Claude API

- Messages API
- Tool use (function calling)
- Vision
- Streaming
- System prompts best practices
- Claude vs GPT-4 differences
- Claude Sonnet vs Opus vs Haiku model selection
- Prompt caching with `cache_control` blocks

- Extended thinking (budget_tokens)
- Computer Use API — screenshot + action loop
- Batch API — async bulk processing

4.4 LiteLLM + NEW

Universal interface for all LLM providers — essential for production multi-provider setups.

- Universal LLM interface — one API for all providers [NEW]
- Provider fallbacks — OpenAI to Claude to Gemini [NEW]
- Cost tracking across providers [NEW]
- LiteLLM proxy server [NEW]
- Load balancing across API keys [NEW]
- Logging + observability integration [NEW]

4.5 Instructor Library + NEW

Most widely used library for getting structured Pydantic outputs from LLMs.

- Structured LLM outputs with Pydantic [NEW]
- Retry on validation failure [NEW]
- Partial streaming with structured data [NEW]
- Works with OpenAI + Claude + Gemini [NEW]
- Field-level validation in LLM responses [NEW]

5.1 LangChain

- LLMChain, SequentialChain
- Prompt Templates — ChatPromptTemplate
- Output Parsers — Pydantic, JSON
- Memory — ConversationBufferMemory, Summary
- Document Loaders — PDF, CSV, Web
- Text Splitters — RecursiveCharacterTextSplitter
- Retrievers
- Tools and Agents
- LCEL — LangChain Expression Language
- Callbacks and streaming

5.2 LangGraph

- State machines for agents
- Nodes and Edges
- Conditional routing
- Human-in-the-loop
- Checkpointing and persistence
- Multi-agent graphs
- Supervisor pattern
- Streaming graph execution
- LangGraph Studio — visual debugging
- Interrupt and resume patterns
- Subgraph composition
- Map-reduce patterns in graphs
- Long-running agent state management

5.3 RAG (Retrieval Augmented Generation)

- Chunking strategies — fixed, semantic, hierarchical
- Embedding models — OpenAI, sentence-transformers
- Vector databases — Pinecone, Chroma, Qdrant, pgvector
- Similarity search — cosine, dot product
- Hybrid search — BM25 + vector
- Reranking — cross-encoder
- Query transformation — HyDE, multi-query
- Contextual compression

- Evaluation — RAGAS framework

5.4 Anthropic MCP (Model Context Protocol)

- MCP architecture — client, server, transport
- Tools — defining, input schema, error handling
- Resources — static and dynamic
- Prompts — reusable prompt templates
- stdio transport
- HTTP + SSE transport
- Authentication in MCP
- Building MCP server in Python (FastMCP)
- Building MCP server in Node.js
- MCP client integration with Claude
- Testing MCP servers
- Production deployment of MCP
- Claude Desktop integration — local MCP testing
- MCP with OAuth — third-party service auth
- Composing multiple MCP servers
- MCP Inspector tool — debugging

5.5 Agent Patterns

- ReAct pattern — Reason + Act
- Plan and Execute
- Reflection pattern
- Multi-agent systems
- Tool selection and routing
- Memory types — short-term, long-term, episodic
- Agent evaluation
- Error recovery in agents

5.6 CrewAI + NEW

Second most popular agent framework — widely used for role-based multi-agent systems.

- Agents — role, goal, backstory [NEW]
- Tasks — description, expected output [NEW]
- Crew — sequential vs hierarchical process [NEW]
- Custom tools in CrewAI [NEW]
- CrewAI vs LangGraph — when to use which [NEW]
- Memory in CrewAI [NEW]

5.7 Agent Memory Systems + NEW

Persistent memory is what separates production agents from demos.

- Mem0 — persistent cross-session memory [\[NEW\]](#)
- Zep — temporal memory for agents [\[NEW\]](#)
- Memory types — facts, preferences, history [\[NEW\]](#)
- Memory retrieval in agent loop [\[NEW\]](#)
- Redis as short-term agent memory [\[NEW\]](#)
- PostgreSQL as long-term memory store [\[NEW\]](#)

5.8 Web Search Tools for Agents + [NEW](#)

- Tavily API — AI-optimized search (LangChain default) [\[NEW\]](#)
- SerpAPI — Google results programmatically [\[NEW\]](#)
- Brave Search API — privacy-focused [\[NEW\]](#)
- Jina AI Reader — URL to clean text [\[NEW\]](#)
- Firecrawl — full website scraping for agents [\[NEW\]](#)
- Building a search tool in MCP [\[NEW\]](#)

5.9 DSPy + [NEW](#)

Emerging paradigm: programming with LLMs instead of manual prompting.

- Programming with foundation models (not just prompting) [\[NEW\]](#)
- Signatures — define input/output [\[NEW\]](#)
- Modules — ChainOfThought, ReAct, ProgramOfThought [\[NEW\]](#)
- Optimizers — BootstrapFewShot, MIPRO [\[NEW\]](#)
- DSPy vs LangChain — philosophy difference [\[NEW\]](#)
- When to use DSPy over manual prompting [\[NEW\]](#)

6.1 Vector Databases Deep Dive

- pgvector — PostgreSQL extension
- Pinecone — namespaces, metadata filtering
- Chroma — local and client-server
- Qdrant — collections, payload filters
- Weaviate — schema, modules
- HNSW index — how it works
- ANN vs exact search tradeoffs
- LanceDB — embedded vector DB (no server needed)
- Redis Vector Search
- Multimodal embeddings — CLIP for image+text
- Sparse vs Dense vs Hybrid index types

6.2 Production AI Systems

- Prompt versioning and management
- A/B testing prompts
- Observability — LangSmith, Langfuse, Helicone
- Guardrails — input/output validation
- PII detection and redaction
- Content moderation
- Caching LLM responses
- Semantic caching
- Fallback models
- Cost monitoring and alerts
- DeepEval — LLM evaluation framework
- TruLens — RAG triad evaluation
- PromptFoo — prompt testing and CI integration
- Structured logging for AI — trace_id, span_id per LLM call
- AI-specific rate limiting — per user token budget
- Response streaming — Server-Sent Events architecture
- Webhook callbacks for async AI jobs

6.3 AI APIs Integration

- Google Gemini API — vision, multimodal
- Gemini Flash for OCR
- Hugging Face Inference API

- Replicate API
- Groq API — fast inference
- Together AI
- Local models — Ollama

6.4 Specialized AI

- Document processing — PDF, tables, forms
- OCR — Tesseract, Google Vision, AWS Textract
- Speech to text — Whisper
- Image generation — DALL-E, Stable Diffusion API
- Web scraping + AI — BeautifulSoup + LLM parsing

6.5 LLM Fine-tuning + NEW

Know WHEN to fine-tune vs RAG — this is a senior-level differentiator.

- When to fine-tune vs RAG — decision framework [NEW]
- OpenAI fine-tuning API — JSONL format, training [NEW]
- LoRA / QLoRA — parameter-efficient fine-tuning [NEW]
- Hugging Face PEFT library [NEW]
- Unsloth — fast fine-tuning [NEW]
- Fine-tuning evaluation — before vs after metrics [NEW]
- When fine-tuning fails — common mistakes [NEW]

6.6 GraphRAG + Knowledge Graphs + NEW

- Microsoft GraphRAG — community summaries [NEW]
- Neo4j — graph database basics [NEW]
- Knowledge graph construction from documents [NEW]
- Graph + Vector hybrid retrieval [NEW]
- Entity extraction for knowledge graphs [NEW]
- LlamaIndex + knowledge graphs [NEW]

6.7 Enterprise AI Platforms + NEW

Where enterprise money flows — knowing these platforms differentiates senior candidates.

- AWS Bedrock — Claude + Titan + Llama via AWS [NEW]
- Google Vertex AI — Gemini via GCP [NEW]
- Azure OpenAI — GPT-4 via Azure [NEW]
- Why enterprises use these over direct APIs [NEW]
- IAM + VPC integration for secure AI [NEW]
- Cost comparison — direct API vs managed platform [NEW]

6.8 AI Testing + NEW

Often ignored but asked in senior interviews — production AI without tests is fragile.

- Mocking LLM calls in tests — pytest + responses [\[NEW\]](#)
- Testing agents deterministically [\[NEW\]](#)
- Snapshot testing for LLM outputs [\[NEW\]](#)
- Regression testing for prompts [\[NEW\]](#)
- Load testing AI endpoints — Locust with LLM [\[NEW\]](#)
- Testing RAG pipeline — retrieval quality metrics [\[NEW\]](#)

PHASE 7: Projects — Build & Deploy

Week 16

Project 1 Personal AI Assistant with MCP

Main flagship project — covers the entire stack end-to-end.

- FastAPI backend
- Multiple MCP tools (file system, web search, database)
- RAG over your documents
- LangGraph for multi-step reasoning
- React frontend with streaming
- Docker + GitHub Actions CI/CD
- Deploy on AWS EC2
- Prometheus + Grafana monitoring

Project 2 RAG Document Q&A; System

Demonstrates deep RAG expertise — a common interview ask.

- Multi-format ingestion (PDF, DOCX, Excel, web URLs)
- Hybrid search (BM25 + pgvector)
- Reranking pipeline
- RAGAS evaluation dashboard
- User feedback loop — thumbs up/down
- Cost tracking per query

Project 3 Multi-Agent Code Review System

Shows multi-agent coordination and GitHub integration skills.

- GitHub webhook integration
- LangGraph supervisor with specialized agents (security, performance, style)
- MCP tool for GitHub API
- Structured review output with Instructor
- Slack notification via MCP

Project 4 Production AI API — Mini SaaS

Shows you can build a real, monetizable AI product.

- Multi-tenant architecture — per-user token budgets
- LiteLLM proxy with provider fallbacks
- Semantic caching with Redis
- Langfuse observability
- Stripe integration for usage billing
- Admin dashboard — cost per user

8.1 System Design for AI Interviews

Every senior AI role will ask at least one of these. Prepare all of them.

- Design a RAG system for 100 million documents
- Design a multi-agent customer support system
- Design LLM cost optimization at scale
- Design a prompt A/B testing system
- Design a real-time AI chat with memory
- Design an AI pipeline with human-in-the-loop

8.2 Coding Patterns to Practice

- Implement ReAct agent from scratch (no framework)
- Build streaming FastAPI + SSE endpoint
- Write a custom LangGraph node with error recovery
- Implement semantic cache with Redis
- Build a chunking strategy for mixed content
- Token counting + cost estimation utility

8.3 Behavioral Questions — Senior Level

- "Tell me about a production AI failure and how you fixed it"
- "How did you reduce LLM costs in a project?"
- "How do you evaluate if a RAG system is working?"
- "Describe a multi-agent system you designed"
- "How do you handle hallucinations in production?"

8.4 Key Technical Concepts to Master

- Explain HNSW index and why it is used in vector databases
- RAGAS metrics — faithfulness vs answer relevancy
- When would you use LangGraph vs raw OpenAI function calls?
- How do you handle hallucination in a production RAG system?
- RAG vs Fine-tuning — decision framework
- Token budget management at scale

Priority Stack — What Gets You Hired

Tier 1 — Non-negotiable (Master First)

1. LangGraph — agent orchestration, dominant pattern
2. RAG pipeline end-to-end — chunking to reranking
3. Tool use / Function calling — OpenAI and Claude APIs
4. MCP (Model Context Protocol) — Anthropic standard
5. Async FastAPI — AI backend is async-heavy

Tier 2 — Strong Differentiator

1. Prompt engineering — CoT, structured outputs, system prompt design
2. Vector databases — pgvector + one cloud option (Pinecone or Qdrant)
3. LangSmith or Langfuse — AI-specific observability
4. Cost optimization — caching, model routing, batching
5. LiteLLM — multi-provider production systems

Tier 3 — Nice to Have

1. LangChain (concepts only — LangGraph supersedes it)
2. CrewAI, DSPy
3. Fine-tuning (know WHEN, not necessarily HOW)
4. GraphRAG, Knowledge Graphs
5. Enterprise platforms — Bedrock, Vertex AI, Azure OpenAI

Resource Stack

Topic	Resource
LangGraph	langchain-ai.github.io/langgraph — official tutorials
RAG	DeepLearning.AI — Building and Evaluating Advanced RAG (free)
Claude API	docs.anthropic.com — tool use section
MCP	modelcontextprotocol.io — FastMCP Python SDK
Pydantic v2	Official Pydantic v2 migration guide
Vector DBs	Qdrant docs — best written docs in the space
Production AI	Eugene Yan's blog on LLM systems
LiteLLM	litellm.ai docs
Instructor	instructor-ai.github.io
DSPy	DSPy official documentation
CrewAI	docs.crewai.com
Fine-tuning	Hugging Face PEFT documentation

10-Week Timeline (Adjusted for 4 Years Experience)

Week 1	Python gaps + AI Foundations (LLM basics, Claude + OpenAI API)
Week 2	Prompt Engineering + Embeddings + pgvector
Week 3	RAG Pipeline — chunking, retrieval, reranking, RAGAS
Week 4	LangGraph — state machines, multi-agent, checkpointing
Week 5	MCP — FastMCP server, tools, resources, deployment
Week 6	Production AI — LangSmith, semantic caching, guardrails
Week 7	Async backend patterns — streaming SSE, Celery + AI

Week 8-9	Build Portfolio Projects (2-3 projects from Phase 7)
Week 10	Interview preparation — system design + coding patterns

Total: ~10 weeks | Compressed from 17 weeks due to existing 4-year backend experience. Your backend expertise is your biggest advantage — most AI engineers can't build production systems.